

Antigravity LMS Master Guide

Antigravity LMS Master Guide: Next.js White-Label Platform

Unified Agent Operating Contract + Phase-by-Phase Frontend Implementation Guide

Next.js (App Router) + TypeScript | Tailwind CSS + shadcn/ui | Server Actions & Dynamic Client Hooks

1. Operating Identity & Next.js Architecture Contract

This specification serves as the formal project contract for an Antigravity coding agent building an enterprise white-label tuition and learning management system.

Each client deployment receives an independent frontend and database instance. Multi-tenancy isolation logic is eliminated. Instead, the focus is on **maximum reusability, modular feature toggles, a zero-hardcoding dynamic permission matrix, and an in-place CMS customizer.**

Architectural Shifts for Next.js App Router

- Rendering Strategy:** Leverage Server Components for layouts, SEO metadata, and initial page shells. Use Client Components (`"use client"`) for interactive state, dynamic permission matrix grids, in-place CMS editors, and real-time quiz countdowns.
- Server-Side File Persistence:** In-place visual edits write directly to the local content configuration (`lib/content-v2.json`) via Next.js Route Handlers (`app/api/admin/content/route.ts`) or Server Actions using standard Node file-system calls (`node:fs/promises`).
- Environment Configuration:** Feature flags are centralized in `config/features.ts` using `NEXT_PUBLIC_` prefixes for client components while securing backend adapters with standard private environment variables.
- Navigation & Guards:** Route Protection uses `middleware.ts` for top-level module isolation alongside client-side `<FeatureGuard>` and `<PermissionGuard>` wrappers.

Non-Negotiable Agent Rules

- Strict Phase Gates:** Never build the entire platform in one prompt. Execute strictly phase-by-phase. Run verification tests and document completion before proceeding.
- No Business Data in Views:** Components must never embed raw fixtures or direct fetch calls. Data must flow through typed repository contracts and custom React hooks.
- Zero Hardcoded Roles:** Prohibit checks like `role === 'teacher'`. All actions evaluate dynamically through permission codes (`module.action`) and contextual scope IDs (`classId` , `batchId`).
- Clean Module Suppression:** When a module flag is disabled, its sidebar links, routes, dashboard cards, and underlying requests must be suppressed.
- Dual-State Repository Factory:** Architecture uses mock repositories by default for local development while matching REST contracts for future backend swap-in.

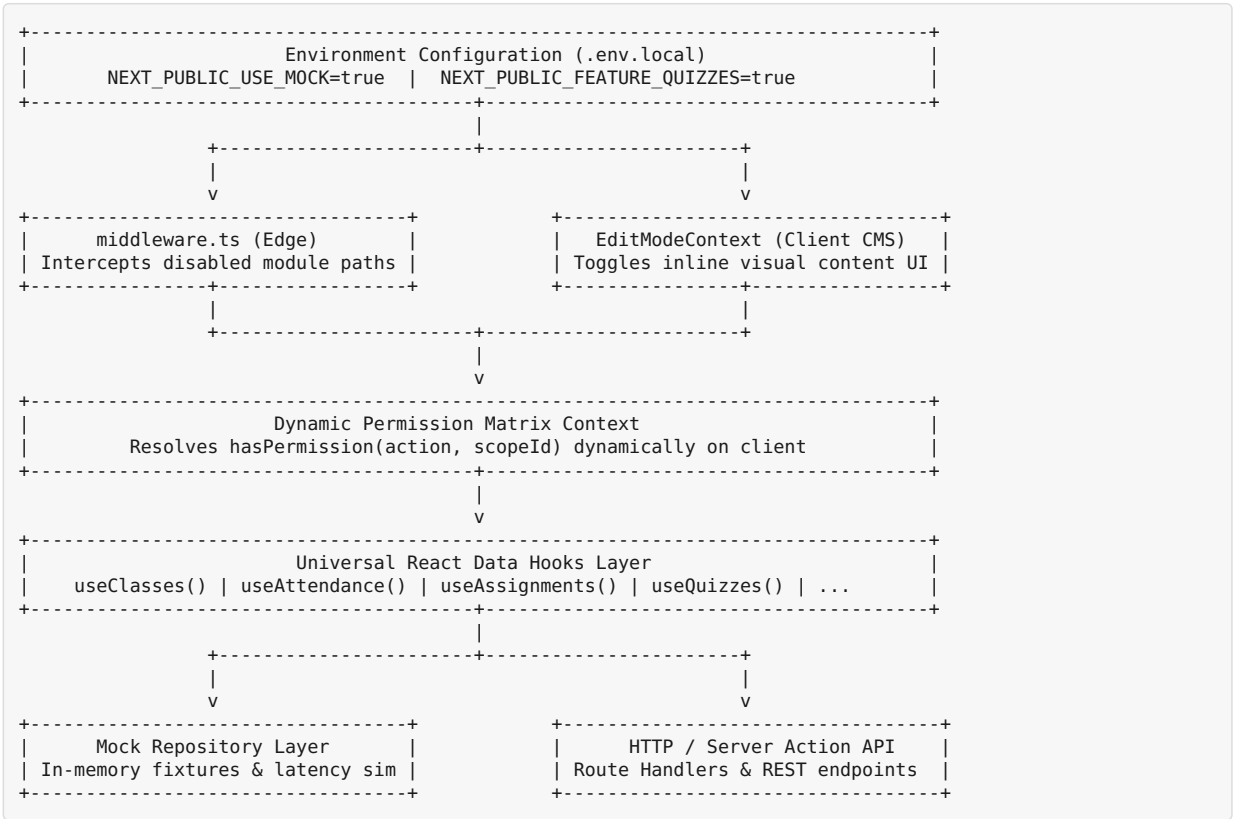
2. Directory & Agent Governance Structure (`.agent/`)

The repository must contain an `.agent/` folder to regulate sub-agent tasks:

- `.agent/AGENT.md` : Master operating instructions, task boundaries, and execution rules.
- `.agent/RULES.md` : UI/UX standards, state management constraints, and error-handling requirements.
- `.agent/WORKFLOW.md` : Phase transition gates and Definition of Done verification rules.
- `.agent/ARCHITECTURE.md` : Server/Client component splits, hook lifecycle patterns, and repository contracts.
- `.agent/REQUIREMENTS_MATRIX.md` : Traceability matrix mapping every UI element to system capabilities.

Recommended sub-agent assignments: **Next.js Architect Agent**, **Permissions & Security Agent**, **Class & Meeting Agent**, **Assessments Agent**, and **UI/QA Test Agent**.

3. High-Level Next.js Architecture



4. Cross-Cutting Systems & Core Patterns

A. Environment Feature Toggles

Feature sets are controlled via environment variables. The configuration module parses these booleans and exposes an immutable features definition:

- `NEXT_PUBLIC_FEATURE_VIRTUAL_MEETINGS` : Live Zoom/Video sessions and cloud recordings.
- `NEXT_PUBLIC_FEATURE_MATERIALS` : Course files, notes, and study material distribution.
- `NEXT_PUBLIC_FEATURE_ASSIGNMENTS` : Homework submissions and evaluation pipelines.
- `NEXT_PUBLIC_FEATURE QUIZZES` : Timed assessments, instant scoring, and practice sets.
- `NEXT_PUBLIC_FEATURE_GAMIFIED_ARENA` : Competitive 1v1 quiz battles and ELO leaderboards.
- `NEXT_PUBLIC_FEATURE_ATTENDANCE` : Automated meeting participation tracking and physical logs.
- `NEXT_PUBLIC_FEATURE_STUDENT_ANALYTICS` : Performance radar metrics and early warning alerts.

Components and layout sections must be wrapped with a `<FeatureGuard feature="KEY">` element. If a feature is disabled, child elements will not mount and routes will redirect to a 404 page.

B. Dynamic Permission Matrix & Scoped Access

- **Atomic Action Strings:** Permissions use explicit codes (`classes.create`, `attendance.mark`, `assignments.grade`, `quizzes.host`).
- **Resource Scoping:** Permissions resolve at three levels:
 1. *Global Scope:* Unrestricted cross-platform capability.
 2. *Class Scope:* Restricted to a specific `classId`.
 3. *Batch Scope:* Restricted to a specific `batchId`.

- **Dynamic Grid UI:** An administrative interface displays permissions where **Rows** represent functional modules and **Columns** represent actions (Create, Read, Update, Delete, Execute). Ticking a box immediately dispatches an update through the permission hook.
- **Component Guard:** A `<PermissionGuard permission="code" scopeId="optional-id">` wrapper renders its children only when access is verified, displaying fallback elements or disabled buttons when denied.

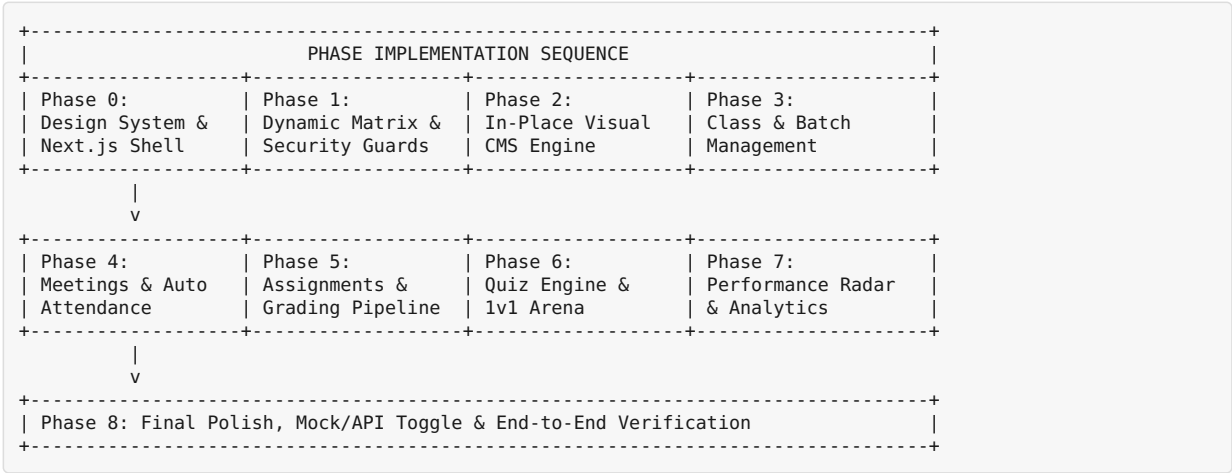
C. In-Place Visual CMS Customizer (Edit Mode)

- **Global Context (`EditModeContext`):** Exposes an `isEditMode` state and a `toggleEditMode` function. This toggle is visible only to users with the `cms.edit` permission.
- **`<EditableContent/>` Wrapper:**
 - *Standard View:* Renders normal typography elements (`h1` , `p` , `span` , etc.).
 - *Edit View:* Displays a dashed boundary with an edit icon on hover. Clicking opens an inline editing box with confirm and cancel buttons.
- **`<EditableImage/>` Wrapper:**
 - *Standard View:* Displays the target image.
 - *Edit View:* Displays an overlay on hover. Clicking launches a modal with zoom, pan, and crop tools to maintain aspect ratio constraints.
- **Configuration Persistence:** Edits are mapped to dot-notation keys (e.g., `dashboard.welcomeBanner.title`) and saved to `lib/content-v2.json` via a Next.js Route Handler (`/api/admin/content`).

D. Data Layer: React Hooks & Dual Repositories

- **Repository Contracts:** Every module exposes a clean TypeScript interface (e.g., `IClassRepository` , `IAttendanceRepository`).
- **Mock Implementations:** Mock stores provide realistic mock data with simulated asynchronous latency, pagination, sorting, and error branches.
- **Service Switch (`repositoryFactory`):** Evaluates `NEXT_PUBLIC_USE MOCK` to bind either mock or HTTP implementations to custom hooks without requiring view-level changes.
- **Custom React Hooks:** Custom hooks manage internal state, cache deduplication, and manual revalidation triggers to minimize unnecessary network requests.

5. Phase-by-Phase Implementation Roadmap



Phase 0: Design System Foundation & Next.js App Shell

- **Exact Scope:**
 - Initialize Next.js App Router with TypeScript and Tailwind CSS.
 - Install and configure shadcn/ui primitives (dialog, button, dropdown-menu, table, badge, toast).
 - Implement environment configuration schema (`config/features.ts`) and `middleware.ts` route protection.

- Build the responsive dashboard layout featuring a collapsible sidebar, breadcrumbs, user profile drawer, and active module links.
 - **Verification Gate:**
 - Toggling `.env.local` feature flags must immediately show or hide corresponding navigation items.
 - The shell must adapt cleanly between mobile screens and desktop monitors without breaking layout flow.
-

Phase 1: Dynamic Permission Matrix & Security Guards

- **Exact Scope:**
 - Build TypeScript contracts for roles, permissions, scopes, and user assignments.
 - Implement `MockPermissionRepository` pre-populated with default roles (Administrator, Instructor, Assistant, Student) and action codes.
 - Implement `usePermissions()` hook providing dynamic authorization evaluation: `hasPermission(action, scopeId?)`.
 - Build the administrative **Permission Matrix Grid Component:**
 - Modular rows with grouped actions (Create, Read, Update, Delete, Special).
 - Checkbox selectors with batch “Select All” actions.
 - Real-time state persistence to the mock repository.
 - Build the `<PermissionGuard/>` component to secure interface actions and display fallback states.
 - **Verification Gate:**
 - Revoking `classes.create` in the Matrix Grid must immediately disable or unmount class creation triggers.
 - Scoped permissions must grant access for an assigned target class ID while denying access for unassigned classes.
-

Phase 2: In-Place Visual Customizer (CMS Engine)

- **Exact Scope:**
 - Implement `EditModeContext` and connect it to `hasPermission('cms.edit')`.
 - Build `<EditableContent/>` supporting dynamic HTML element rendering (`as` prop), plain text, sanitized HTML, and inline input transitions.
 - Build `<EditableImage/>` with modal dialog integration, image file reading, and aspect-ratio-locked cropping via `react-easy-crop`.
 - Implement Route Handler `app/api/admin/content/route.ts` to persist edits to `lib/content-v2.json`.
 - **Verification Gate:**
 - Activating Edit Mode must visually identify all customizable components via dashed hover outlines.
 - Modifying text inline and cropping a replacement image must persist to `lib/content-v2.json` and survive page refreshes.
-

Phase 3: Class & Batch Management Module

- **Exact Scope:**
- Build contracts, mock fixtures, and `useClasses()` hook for courses, batches, and enrollment tracking.
- Build Class Catalog and Detail Views showing batch schedules (Theory, Revision, Seminar), pricing, and capacities.

- Build class creation and edit modal forms guarded by `classes.create` and `classes.update`.
 - Build student enrollment application forms and a moderation dashboard to approve or reject pending requests.
 - **Verification Gate:**
 - Submitting an enrollment application must add the student to the pending approval queue.
 - Approving the application updates the student's status to active and increments batch enrollment counts.
-

Phase 4: Virtual Meetings & Automated Attendance Engine

- **Exact Scope:**
 - Define data structures for meetings and attendance sessions (`PRESENT` , `ABSENT` , `LATE`).
 - Implement `useAttendance()` hook handling session logs and manual record mutations.
 - Build Live Meeting Launcher displaying status badges, countdowns, and Join/Start buttons guarded by permission.
 - Build the Attendance Tracking Workspace:
 - Interactive student attendance grid supporting one-click status overrides.
 - Webhook simulation tool that parses attendee participation logs and marks students present if their duration meets threshold percentages.
 - **Verification Gate:**
 - Running the webhook simulation must automatically mark students present or late based on elapsed minutes.
 - Manual instructor overrides on the attendance grid must update batch metrics immediately.
-

Phase 5: Assignments & Rubric Evaluation Pipeline

- **Exact Scope:**
 - Define data models for assignments, file submissions, and multi-criteria scoring rubrics.
 - Implement `useAssignments()` hook supporting student submission states and evaluation queues.
 - Build Student Homework Portal with file upload drop zones, external link submissions, and deadline indicators.
 - Build Evaluator Workspace:
 - Submission listing with status filters (`SUBMITTED` , `EVALUATED` , `LATE`).
 - Grading drawer featuring split-view document preview, rubric scoring inputs, and structured feedback forms.
 - **Verification Gate:**
 - Submissions past the configured due date must automatically display a `LATE` badge.
 - Submitting a completed rubric evaluation must compute the total score and display feedback in the student view.
-

Phase 6: Modular Quiz Engine & Lightweight 1v1 Arena

- **Exact Scope:**
- Define models for quizzes, questions, powerups, match sessions, and ELO ratings.

- Implement `useQuizzes()` hook managing timed question queues and score computations.
 - Build Single-Player Quiz Interface featuring animated countdown timers, progress bars, and instant answer reviews.
 - Build lightweight powerup mechanics:
 - `fifty_fifty`: Hides two incorrect multiple-choice options.
 - `extra_time`: Adds 20 seconds to the active timer.
 - Build Asynchronous 1v1 Challenge Arena:
 - Matchmaking lobby displaying student leaderboards and active challenge invitations.
 - Challenge execution screen comparing final scores and updating ELO rating adjustments (ΔELO).
 - **Verification Gate:**
 - Triggering the `fifty_fifty` powerup must reliably hide two incorrect choices for the active question.
 - Completing a 1v1 challenge must record scores for both players and calculate correct rating adjustments.
-

Phase 7: Student Performance Radar & Analytics

- **Exact Scope:**
 - Define models for aggregated metrics: attendance percentages, homework completion, quiz averages, and ELO ranks.
 - Implement `useAnalytics()` hook aggregating cross-module records into unified student performance profiles.
 - Build Performance Radar Chart visualizing multi-axis competencies (Consistency, Accuracy, Speed, Homework).
 - Build Early Intervention Table highlighting students falling below configurable thresholds (e.g., $< 60\%$ attendance or missing assignments).
 - **Verification Gate:**
 - A mock student with low attendance and missing assignments must be automatically flagged with an **At-Risk Warning** badge.
 - The performance radar chart must reflect changes made in the assignment and quiz modules.
-

Phase 8: Hardening, Repository Switch & Integration Review

- **Exact Scope:**
 - Verify client adaptability by toggling `.env.local` feature flags and confirming interface stability.
 - Verify `NEXT_PUBLIC_USE MOCK=false` redirects data hook requests to target API contracts with expected payloads and headers.
 - Build JSON Content Exporter to package all in-place CMS customizations into a clean deployment payload.
 - Perform accessibility, layout, and keyboard-navigation quality audits across all modules.
 - **Verification Gate:**
 - Project must compile cleanly (`tsc --noEmit` and `next build`) without TypeScript or runtime errors.
 - All views must display explicit loading, empty, and error fallback states across simulated failure scenarios.
-

6. Definition of Done & Phase Sign-Off Template

Every phase must pass all criteria below before the next phase begins:

1. **Requirements Traced:** Every interface element maps directly to module specifications.
2. **Access & Features Guarded:** Permissions and feature flags are strictly enforced.
3. **Data Layer Decoupled:** Page views interact exclusively with custom React hooks.
4. **State Coverage:** Loading skeletons, error warnings, and empty placeholders are fully implemented.
5. **Phase Tests Verified:** All test criteria in the verification gate pass without regressions.

Phase Completion Report Format

When concluding an implementation phase, the agent must output a structured sign-off report:

```
### Phase Completion Sign-Off
- **Phase Completed**: [e.g., Phase 3 - Class & Batch Management]
- **Delivered Capabilities**: [List of built views, drawers, modals, and hooks]
- **Files Added/Updated**: [Relative paths to newly created or modified files]
- **Permissions Implemented**: [List of permission action strings registered]
- **State Validation Summary**: [Confirmation of loading, error, and empty states]
- **Test Gate Verification**: [Itemized confirmation of Phase Verification Checklist]
- **Open Questions / Assumptions**: [Any structural assumptions made during execution]
- **Next Phase Prerequisite**: [Dependencies verified before beginning subsequent phase]
```